

Eurobas API

What the Flutter app needs from the API for feature parity

This document lists what the mobile app needs from the API (routes/api/v1/api.php) to reach parity with the features that surface on the live website. Legend:  = working,  = exists but missing fields/behaviour,  = no API yet.

The headline: read-only browsing is well covered. The two biggest holes are that a user **cannot post/edit/delete a listing** and **cannot pay for any promotion** through the API — that functionality exists only on the website today. Those two gaps alone block a feature-complete app.

1. Summary

Area	Status	Note
Auth, OTP verify, password reset		working
Social login (API endpoint)	 / 	endpoint works; Google & Facebook blocked on credentials — see §7
Browse / search / filter ads		POST ads/filter is strong; minor fixes on by-category & home feed
Ad detail		missing auction/asking-price offer lists; needs slug lookup for deep links
Post / edit / delete a listing	 CRITICAL	no API exists
Promotions & payments (paid banners, boost/sponsor, PayPal/Stripe, Mux video)	 CRITICAL	no payment/promotion API exists

Area	Status	Note
Offers (auction bid / asking-price)	⚠ / ✗	auction API incomplete; asking-price has no API
Report ad	✗	no API
Chat / messaging	⚠	text-only: no images, no ad-context, no unread count, no mark-seen
Wishlist	✓	missing only "clear all"
Public seller profile	⚠	missing ads list, categories, brands, postal/street
Brands / vehicle-models browsing	✗	no API
Support tickets	⚠	no delete, no attachments
Static pages (terms/privacy/about/...)	⚠	only 3 of 6 available, via /config
Config endpoint	⚠	customer_image_url is wrong; no payment-gateway flags
Profile / cover image upload	✗	API is broken — see §6.5
Google Maps keys	✗	provided keys don't work — see §7

2. What already works (no backend work needed)

Feature	API endpoint
Register (individual/company)	POST auth/register
Login / logout / refresh	POST auth/login, auth/logout, auth/refresh
Phone OTP: check / verify / resend	POST auth/check-phone, verify-phone, resend-otp-check-phone
Email OTP: check / verify / resend	POST auth/check-email, verify-email, resend-otp-check-email
Social login (token verified server-side)	POST auth/social-login, auth/update-phone
Forgot/reset password	POST auth/forgot-password, resend-otp-reset-password, verify-otp,

Feature	API endpoint
	GET auth/verify-email-token, PUT auth/reset-password
Get / update own profile (text fields + password)	GET customer/profile, PUT/POST customer/profile/update
Account info, delete, FCM token	GET customer/info, customer/account-delete, PUT customer/cm-firebase-token
Addresses (add/update/list/get/delete) + restricted country/zip	customer/address/*, customer/get-restricted-*
Browse / filter ads (incl. geo-radius)	POST ads/filter
Search ads	POST searched-ads
My ads	GET/POST customer/profile/ads
Wishlist (list/add/remove)	GET customer/wish-list, POST .../add, DELETE .../remove
Public user (visibility-aware)	GET users/{id}
Categories, popular categories, banners, paid-banners (read)	GET categories, categories/popular-categories, banners, paid-banners
Translations (+ version polling)	GET/POST locale/translations/{locale}, .../version
Map proxy (autocomplete/details/geocode/distance)	GET mapapi/*
Contact, FAQ, newsletter, social links, config, guest-id	POST contact-us, GET faq, POST subscription, GET social-media, config, get-guest-id
Support tickets: create/list/conv/reply/close	customer/support-ticket/*
Chat: list / messages / send (text)	customer/chat/*

3. CRITICAL gap #1 — Listing lifecycle (post / edit / delete)

A user **cannot create, edit, or delete an ad** through the API. The website does all of this via Web/AdController (store, edit, update, delete), but api.php exposes only read endpoints plus POST ads/auction. This is the single biggest blocker to a feature-complete app.

The website posting flow (Web/AdController@store, ~:194-348) collects ~60 fields: core (title, description, category_id, brand_id, model_id, custom_brand), pricing

(`price_type ∈ fixed_price | asking_price | auction | free`, `price`, `starting_price`, `allow_offers`, `first_price`, `currency` — `free` and `auction` hide the `price` field; `asking_price` is the "negotiable / make an offer" type), `location` (`country`, `city`, `postal_code`, `latitude`, `longitude`), `contact/visibility` (`show_phone_number`, `show_email_address`, `whatsapp_availability`, `phone_code`, `contact_phone_number`), `media` (`image_thumbnail`, `images[]` gallery, optional Mux `video_id`), and a large category-dependent attribute set (vehicle: `year`, `mileage`, `fuel_type`, `engine_size`, `transmission_type`, `body_type`, `doors_number`, `seats_number`, ...; real estate: `property_type`, `property_size`, `floor`, `rooms_number`; plus furniture / electronics / industrial / marine / battery / dimensions / environmental blocks). Vehicle categories require `year`. Optional sponsor packages can be attached at creation, which branches into the payment flow (§4).

Endpoints we need

- **POST v1/ads** (auth, multipart) — create. Request = the full `store()` field set above; `image` + `images[]` as files. Response = the created ad, plus a payment branch when paid packages are attached.
- **PUT/POST v1/ads/{id}** (auth, owner-scoped) — update. Same fields as `update()` (`AdController.php:490-644`) plus `old_images[]` to retain existing gallery images.
- **DELETE v1/ads/{id}** (auth, owner-scoped) — delete.
- **GET v1/ads/create-options?category_id=** (auth) — form bootstrap: categories filtered by `category_type`, brands (+category mappings), vehicle models (+category mappings), the per-category attribute field definitions and their allowed values, active sponsor packages, and image/video limits (`ad_images_size`, `maximum_ad_images_number`, `maximum_promotional_video_duration/size`). Mirrors web `adding_type` + `add`.

The attribute fields are **category-dependent** — both the set of fields and their allowed values change per category (the website builds them from `List / ListValue` records, e.g. `Web/AdController.php:118-119,179`). So the API must return the per-category field definitions and allowed values, either inside `create-options` or as a separate `GET v1/categories/{id}/fields`.

Why this matters: with no such endpoint, we have had to replicate these field definitions inside the app itself. That couples the app to the backend's current fields — if you add, remove, or change a category's fields, the change will **not** reach users until we rebuild and resubmit the app to the App Store / Play Store, and the two surfaces will silently drift out of sync until then. A per-category fields endpoint removes that coupling, so your backend changes take effect on mobile immediately with no app release.

4. CRITICAL gap #2 — Promotions & payments

The entire paid surface exists only on the website. The app can *read* promotional results but cannot *buy* anything. Payments are in **EUR** via **PayPal + Stripe**. The website payment flow is session- and browser-redirect-based, so a mobile client cannot reuse it; we need JSON API endpoints instead. Web controllers: `Web/PaidBannerController`, `Web/SponsorController`, `Payment_Methods/PaymentController`, `Services/{PaypalPayment,StripePayment}`, `MuxApiController`.

Two paid product families:

- **Paid banners** — upload a banner image, optionally link to one of your ads + a category, pick a paid `SubscriptionPackage` (type `promotional_banner`), pay → it shows on the home page.
- **Sponsors (boost a listing)** — pick an ad + a sponsor type (`urgent_sale_sticker`, `appearance_in_first_results`, `promotional_video`) + a matching paid package, pay → the ad gets the promotion. `promotional_video` additionally needs a `video_id` from a (free) Mux upload.

Endpoints we need

1. **GET `v1/subscription-packages?type=...`** — package catalogue (prerequisite for any purchase). Per item: `id`, `type{id,name}`, `price` (EUR), `duration_in_days`, `status`, `features[]` (e.g. `show_on_home_page`).
 2. **POST `v1/customer/paid-banners`** (multipart) — `banner_image(file)`, `category_id`, `package_id`, `redirect_to_ads(bool)`, `ad_id(nullable)`. Returns the banner id + a payment object when the package price > 0.
 3. **POST `v1/customer/sponsors`** — `ad_id`, `type`, `package_id`, `video_id` (only for `promotional_video`). Returns the sponsor id + a payment object.
 4. **GET `v1/customer/sponsors` + DELETE** for both paid-banners and sponsors (manage my promotions).
 5. **POST `v1/payment/initiate`** — `{model_type: paid_banner|sponsor, model_id, method: paypal|stripe}` → returns a self-contained `{checkout_url}` we can open in a webview/browser (must not depend on a web session). Plus **GET `v1/payment/verify`** (or a server callback) → `{is_paid, payment_transaction_id}`.
 6. **`v1/mux/*`** — the promotional-video upload available via the authenticated API, returning a `video_id` we can pass into the sponsor create call. The upload itself is free.
 7. **Which gateways are enabled** — `/config` does not say whether PayPal/Stripe are active. We need a `payment_gateways` flag in `/config` (or a small payment-config endpoint).
-

5. Offers & report (ad actions)

On the website, **auction bids and asking-price offers are chat-coupled**: placing one writes the AdAuction/AdAskingPrice row *and* a chat message to the seller (via Web/ChattingController@messages_store), so the seller sees the offer in their inbox.

Action	Web	API	What we need
Place auction bid	messages_store (make_an_offer)	⚠ POST ads/auction	It creates the AdAuction row but not the seller-facing chat message, so the seller never sees the bid. We need the bid to also create that chat message (parity with web).
Delete own auction bid	AdController@delete_auction	✗	DELETE v1/ads/auction/{id}.
Place asking-price offer	messages_store (asking_price)	✗	POST v1/ads/asking-price (ad_id, price) that creates the row + chat message.
Delete own asking-price offer	AdController@delete_asking_price	✗	DELETE v1/ads/asking-price/{id}.
Report ad	AdController@report_ad	✗	POST v1/ads/report (id, message).

Also: GET ads/show/{ad} needs to include the auctions and asking_price offer lists (bidder name, price, created_at) so the detail page's offer panels can render — they show on the website but are absent from the API response.

6. Other parity gaps

6.1 Chat is thinner than the website ⚠

The website chat supports image attachments, an "about this ad" context card, per-conversation unread counts, a global unread badge, and auto-mark-seen on open. The API (customer/chat/*) is text-only with no read state. We need:

- **Image attachments** — POST customer/chat/send-message to accept multipart image[] and store/return them; get-messages to return the attachments.
- **"Chat about this ad" context** — an optional ad_id on send-message that is persisted, and the ad reference (id/slug/title/thumbnail) returned by get-messages, so a conversation can show which ad it's about.
- **Unread count** — GET customer/chat/unread-count, plus an unseen_message_count (and a last-message preview/time) per row in chat/list.
- **Mark-seen** — get-messages/{id} to mark that partner's messages seen, or a POST customer/chat/mark-seen.

6.2 Public seller profile is incomplete ⚠

GET users/{id} returns the visibility-aware profile but omits sections the website profile (UserProfileController@show_profile) shows:

- **ads list** — biggest gap. Either add GET users/{id}/ads (paginated, shaped like ads/filter rows) **or** make POST ads/filter honour user_id/profile_id (the website filter already does; the API one ignores it).
- **categories** array (the seller's "we publish in these categories" grid).
- **brands** array (the seller's "and these brands" logos row).
- **postal_code + street_address** (shown on the website, gated by show_location_data).
- **ads_count** — the count returned by GET users/{id} should match the count the website's profile page shows for the same user (they currently differ).

6.3 Browse / discovery ⚠/✗

- **Home feed** — HomeController@index builds per-category ad rows (sponsored-first)
 - banners; there is no equivalent API. categories/popular-categories returns categories with counts but not the ad rows. We need a GET v1/home aggregate, plus a per-category "load more" (web load_home_ads, AdController.php:1719).
- **Category ordering** — the website orders the home categories using UserCategoryInterest (an aggregate "trending categories" signal). For visual parity the home/feed endpoint should apply the same ordering.
- **GET ads/by-category/{id} ⚠** — returns inactive ads (no active filter), with no pagination and no sponsor ordering; needs to match the website's by-category page.
- **Filter result count ⚠** — the website shows a live "N results" count before applying a filter (ads_filter_count, AdController.php:1501). We can read total from POST ads/filter, but a POST v1/ads/filter-count (same params, count only) matches the UX without fetching a page of rows.
- **Brands / vehicle-models browsing ✗** — the website has all_brands, brand_details (a brand's models), and model_ads. No API. We need GET

v1/brands, GET v1/brands/{id} (its models), and a combined brands+models dataset for the cascading category→brand→model filter. Brands must be **per-category and uncapped** — /config only returns the top 15, which is not enough for the post-ad or filter flows.

6.4 Smaller gaps

- **Wishlist clear-all** ❌ — the website has delete-wishlist-all; no API. Add DELETE v1/customer/wish-list/clear.
- **Support ticket delete** ❌ — the website has it; the API only closes. Add DELETE v1/customer/support-ticket/{id}.
- **Support ticket attachments** ⚠️ — the website accepts image[] on create + reply (max 6 MB); the API ignores attachments. Add multipart support to support-ticket/create and reply/{id}.

6.5 Profile / cover image upload — API is broken ❌

PUT/POST customer/profile/update validates image as a string and never reads the uploaded file, so avatar uploads are silently dropped, and cover_image is not writable over the API at all — **this part of the API is currently broken**. The endpoint needs to accept multipart image and cover_image files (stored to profile/images/ and profile/covers/). We've put up a rough PR you're welcome to use as a starting point — it hasn't been thoroughly reviewed on our side and may need changes:

<https://github.com/EuroBas-tech/eurobas-marketplace/pull/17>.

6.6 Config & static content ⚠️

- **base_urls.customer_image_url is wrong** — /config returns .../storage/app/public/profile, which 404s for avatar/cover files. We need /config to return the correct base URL for profile images, and a separate base URL for cover images.
- **Static pages** — /config exposes only about_us, privacy_policy, terms_&_conditions. The website also has instructions-for-use, return-policy, and cancellation-policy, with no API source. We need GET v1/pages/{slug} → {title, content} covering all of them.

6.7 Translation keys needed for parity ⚠️

The translations API works; these keys are simply not yet in the bundle, so the app can't display the corresponding website text. Please add them to the dictionary:

- **Relative join time** — the website shows e.g. "Joined Mar, 2026 (1 month ago)". We need keys for the "... ago" suffix: joined_today, joined_yesterday, joined_n_days_ago, joined_n_months_ago, joined_n_years_ago (with a :count placeholder).
- **view_profile** — the bundle has show_profile but not view_profile, which the seller-card link needs.

6.8 Deep links — ad lookup by slug ⚠

The website's canonical ad URL is by **slug**, but GET `ads/show/{ad}` is by numeric **id**. For shared links / deep links to open the correct ad in the app, we need a slug-based lookup — either GET `v1/ads/show-by-slug/{slug}` or accept a slug on the existing endpoint.

7. Platform credentials we need from you

These are not API changes — they are credentials/keys that only the account owners can create and hand to us.

7.1 Social login (Google & Facebook)

Apple Sign-In is done — we were added to the Apple Developer account and completed it. **Google and Facebook are blocked**: we are not members of your Google Cloud or Facebook developer accounts, so someone on your side with access must create the credentials below and send them to us (or add us to those accounts).

Google Sign-In — work in the Google Cloud project the backend already uses (do not create a new one). In **APIs & Services** → **Credentials**, create OAuth 2.0 client IDs:

- **iOS** — bundle ID `com.eurobas.app`.
- **Android** — package `com.eurobas.app`, plus the SHA-1 of every signing key:
 - `0C:DB:4C:1C:A4:DF:6E:99:89:7F:C0:C4:FE:B3:83:A8:E0:51:89:7F` (debug)
 - `E9:8F:2E:78:73:F1:F6:1F:F2:A2:40:88:3C:8D:BA:A9:80:52:AC:A5` (debug)
 - `B2:AA:0F:1E:6C:AE:78:28:B2:B2:04:31:91:D4:50:53:34:21:32:AB` (release)
- **Web application** — this is the `serverClientId` used by Android.
- **Send us**: the **iOS client ID** and the **Web client ID**.

Facebook Login — at <https://developers.facebook.com>, create/open a **Consumer** app for Eurobas:

- **App settings** → **Basic**: the **App ID** and (under Advanced → Security) the **Client Token**.
- Add the **iOS platform** with bundle ID `com.eurobas.app`.
- Add the **Android platform** with package `com.eurobas.app` and the key hash(es) we provide (computed from our signing keys):
 - `DNtMHKTfbpmJf8DE/rODqOBRiX8=` (debug)
 - `6Y8ueHPx9h/yokCIPi26qYBSrKU=` (debug)
 - `sQoPHmyueCiysgQxkdRQUzQhMqs=` (release)

- Under **Use Cases** → **Customize**, enable **Facebook Login** with the default email and public_profile permissions.
- **Send us:** FACEBOOK_APP_ID and FACEBOOK_CLIENT_TOKEN.

7.2 Google Maps API keys

The Android and iOS Google Maps keys shared earlier **do not work**. As a temporary stopgap the app is currently running on the developer's **own personal Google Maps keys** — these must be replaced with EuroBas's own keys before release (billing and ownership belong to you, not the developer). We need working, correctly-restricted Google Maps API keys for both platforms:

- **iOS** — bundle ID `com.eurobas.app`.
- **Android** — package `com.eurobas.app`, restricted to these signing SHA-1s:
 - `0C:DB:4C:1C:A4:DF:6E:99:89:7F:C0:C4:FE:B3:83:A8:E0:51:89:7F` (debug)
 - `E9:8F:2E:78:73:F1:F6:1F:F2:A2:40:88:3C:8D:BA:A9:80:52:AC:A5` (debug)
 - `B2:AA:0F:1E:6C:AE:78:28:B2:B2:04:31:91:D4:50:53:34:21:32:AB` (release)

8. Suggested priority for the backend team

1. **Listing CRUD** — POST/PUT/DELETE `v1/ads` + GET `v1/ads/create-options`. (§3)
2. **Monetization** — package catalogue, create paid-banner, create sponsor, payment initiate/verify, Mux under auth. (§4)
3. **Offers & report** — asking-price API, complete the auction API (chat-coupled), offer-delete, report-ad, offer lists in ad detail. (§5)
4. **Public profile** — ads list (or `profile_id` on filter), categories, brands, postal/street; reconcile `ads_count`. (§6.2)
5. **Profile/cover image upload** — deploy the fix PR. (§6.5)
6. **Chat** — images, ad-context, unread count, mark-seen. (§6.1)
7. **Browse** — fix `ads/by-category`, brands/models endpoints (per-category, uncapped), category ordering, filter-count, home feed. (§6.3)
8. **Hygiene** — `customer_image_url`, static-page endpoint, payment-gateway flags in `/config`, wishlist clear-all, ticket delete/attachments, slug lookup for deep links, translation keys. (§6.4, §6.6, §6.7, §6.8)